

# Machine Learning Algorithms Laboratory

## Experiment 9 Report

|                  |                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------|
| Experiment No.   | 9                                                                                                    |
| Experiment Title | Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning<br><a href="#">1</a> |
| Student Name     | Gautham S                                                                                            |
| Register Number  | 3122247001016                                                                                        |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                                                         |
| Submission Date  |                                                                                                      |

## 1 Aim and Objective

The aim of this experiment is to implement a Single-Layer Perceptron (PLA) from scratch and compare it against a Multilayer Perceptron (MLP) on a handwritten character recognition task. Hyperparameters for the MLP such as activation function, optimizer, learning rate, batch size and hidden layer architecture are tuned systematically, and the final tuned MLP is compared against the PLA using standard classification metrics.

## 2 Theory

### 2.1 Perceptron Learning Algorithm (PLA)

The Perceptron uses a step activation function and updates its weights whenever it misclassifies a sample, using the rule:

$$w_{t+1} = w_t + \eta(y - \hat{y})x \quad (1)$$

Since a plain Perceptron only produces a binary output, one Perceptron was trained per class (one-vs-rest) to handle this 62-class problem. The key limitation of PLA is that it can only find a linear decision boundary, so it performs poorly when classes are not linearly separable, which is often the case for image data like handwritten characters.

### 2.2 Multilayer Perceptron (MLP)

An MLP has one or more hidden layers between the input and output, with non-linear activation functions (ReLU, Tanh etc.) at each hidden layer. This allows it to learn non-linear decision boundaries through backpropagation, unlike a plain Perceptron. It is trained by minimizing a loss function (cross-entropy for multi-class classification here) using an optimizer such as SGD or Adam.

## 3 Dataset Description

| Dataset           | Description                                           |
|-------------------|-------------------------------------------------------|
| Dataset Name      | English Handwritten Characters Dataset                |
| Number of Samples | 3410 images                                           |
| Number of Classes | 62 (digits 0-9, uppercase A-Z, lowercase a-z)         |
| Image Size        | Resized to 28x28 grayscale, flattened to 784 features |
| Class Balance     | Roughly balanced, about 55 images per class           |
| Train-Test Split  | 80% Train (2728 samples), 20% Test (682 samples)      |

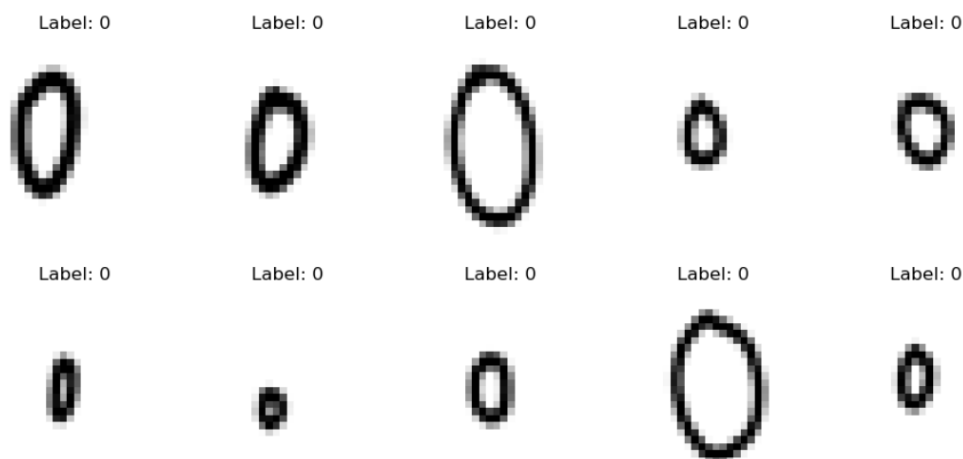


Figure 1: Sample Handwritten Characters from the Dataset

**Inference:** The characters have some variation in stroke thickness and slight rotation/style differences even within the same class, and several characters look visually similar to each other (for example uppercase "O" and digit "0", or "l" and "1"), which can make this a genuinely difficult classification problem, especially with a small number of training images per class.

## 4 Preprocessing Steps

Each image was resized to 28x28 pixels, converted to grayscale, and flattened into a 784-length feature vector. Pixel values were normalized to the 0-1 range. The class labels (originally folder/character names) were encoded into numeric labels using `LabelEncoder`. The data was split into 80% training and 20% testing.

```
img = Image.open(path).convert("L").resize((28, 28))
img_array = np.array(img) / 255.0

X = images.reshape(images.shape[0], -1)    # flatten to 784
      features

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

## 5 PLA Implementation and Results

A Perceptron class was implemented from scratch using the step activation function and the standard weight update rule. Since PLA only supports binary classification, one Perceptron was trained per class in a one-vs-rest manner (62 separate binary Perceptrons), and at prediction time the class whose Perceptron gives the highest score is chosen.

```
class Perceptron:
    def __init__(self, lr=0.01, epochs=40):
        self.lr = lr
        self.epochs = epochs
        self.errors = []

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        for epoch in range(self.epochs):
            error_count = 0
            for xi, target in zip(X, y):
                y_hat = 1 if (np.dot(xi, self.w) + self.b) >= 0
                else 0
                update = self.lr * (target - y_hat)
                self.w += update * xi
                self.b += update
                error_count += int(update != 0.0)
            self.errors.append(error_count)
```

| Model | Accuracy | Precision | Recall | F1-Score |
|-------|----------|-----------|--------|----------|
| PLA   | 0.1364   | 0.1768    | 0.1364 | 0.1025   |

Table 2: PLA Performance on Test Set

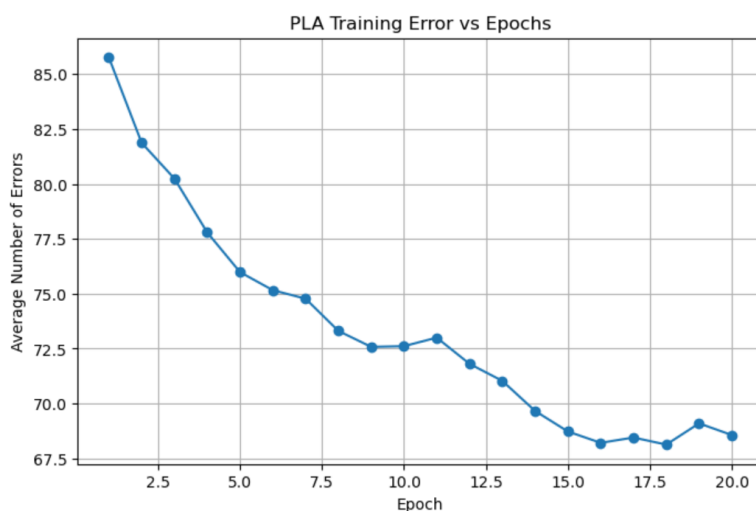


Figure 2: PLA Average Training Error vs Epochs

**Inference:** The PLA’s average number of errors decreases over the 40 training epochs but does not reach zero, which confirms that the 62 classes are not linearly separable in the raw 784-pixel feature space, PLA can only draw a straight decision boundary and simply cannot perfectly separate handwritten character shapes this way. The final test accuracy of only 13.64% (compared to about 1.6% for random guessing among 62 classes) shows PLA does learn something, just not nearly enough for this task.

## 6 MLP Implementation and Hyperparameter Tuning

### 6.1 Baseline MLP

A baseline MLP was first trained using scikit-learn’s default settings (a single hidden layer) with no tuning, just to have a starting point for comparison.

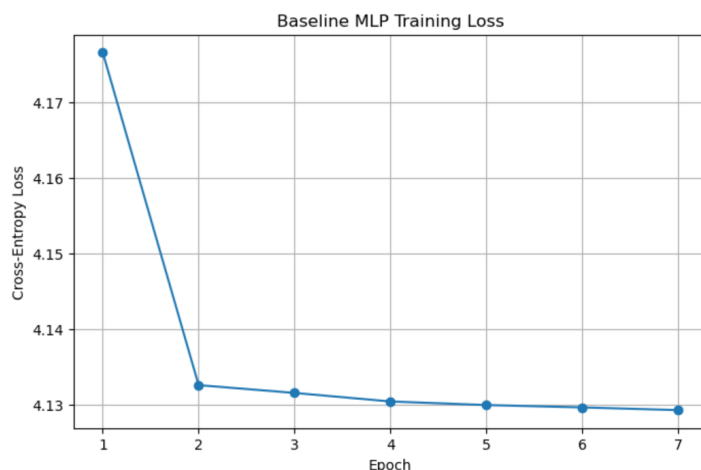


Figure 3: Baseline MLP Training Loss Curve

| Model        | Accuracy | Precision | Recall | F1-Score |
|--------------|----------|-----------|--------|----------|
| Baseline MLP | 0.0132   | 0.0004    | 0.0132 | 0.0008   |

Table 3: Baseline (Untuned) MLP Performance

**Inference:** Surprisingly, the untuned baseline MLP performed even worse than PLA, with an accuracy close to random guessing. It stopped after only 7 epochs with a final training loss of about 4.13, which is very close to  $\ln(62) \approx 4.13$ , the loss value you would expect from a model that is essentially guessing uniformly at random across the 62 classes. This shows that a neural network with poor default hyperparameters (an unsuitable learning rate/architecture combination, stopping too early) can actually perform worse than a much simpler model, tuning is not optional for MLPs, it is necessary for them to work well at all on a task like this.

## 6.2 Hyperparameter Search

Different combinations of activation function, optimizer, learning rate and batch size were tried (with a single hidden layer) to see their effect on accuracy.

```
configs = [
    ("relu", "adam", 0.001, 32), ("relu", "adam", 0.01, 32),
    ("relu", "sgd", 0.001, 32), ("relu", "sgd", 0.01, 32),
    ("tanh", "adam", 0.001, 32), ("tanh", "adam", 0.01, 32),
    ("relu", "adam", 0.001, 64), ("relu", "adam", 0.01, 64),
]

for act, opt, lr, batch in configs:
    print(f"Training: {act}, {opt}, lr={lr}, batch={batch}")
    mlp = MLPClassifier(hidden_layer_sizes=(100,), activation=act
                        ,
                        solver=opt, learning_rate_init=lr,
                        batch_size=batch, max_iter=40,
                        random_state=42)
    mlp.fit(X_train, y_train)
```

| Activation | Optimizer | Learning Rate | Batch Size | Epochs | Accuracy | Time (s) |
|------------|-----------|---------------|------------|--------|----------|----------|
| ReLU       | SGD       | 0.001         | 32         | 40     | 0.2507   | 1.42     |
| Tanh       | Adam      | 0.001         | 32         | 40     | 0.2331   | 2.18     |
| ReLU       | Adam      | 0.001         | 64         | 40     | 0.0572   | 1.29     |
| ReLU       | Adam      | 0.001         | 32         | 32     | 0.0161   | 1.85     |
| ReLU       | SGD       | 0.010         | 32         | 27     | 0.0161   | 0.92     |
| ReLU       | Adam      | 0.010         | 32         | 27     | 0.0161   | 1.54     |
| Tanh       | Adam      | 0.010         | 32         | 20     | 0.0161   | 1.10     |
| ReLU       | Adam      | 0.010         | 64         | 27     | 0.0161   | 0.84     |

Table 4: Activation/Optimizer/Learning Rate/Batch Size Search (single hidden layer)

**Inference:** A learning rate of 0.01 was clearly too large for this data, every single configuration using  $lr=0.01$  collapsed to an accuracy of about 0.0161 (basically random guessing), likely because the weights were updated too aggressively and the model never settled into a good solution. A smaller learning rate of 0.001 combined with ReLU and SGD gave the best result in this stage (0.2507), noticeably better than both PLA and the untuned baseline MLP.

## 6.3 Architecture Search

Different hidden layer architectures were then compared using the best learning rate found above.

| Hidden Layers | Architecture | Accuracy |
|---------------|--------------|----------|
| 1             | (128,)       | 0.0161   |
| 2             | (128, 64)    | 0.3284   |

Table 5: Effect of Number of Hidden Layers

**Inference:** Adding a second hidden layer made a massive difference here, accuracy jumped from 0.0161 (a single (128,) layer, which collapsed just like the poorly tuned

configurations above) to 0.3284 with a (128, 64) two-layer architecture. This particular single-layer run likely also got stuck in a bad optimization region, similar to what was seen with a high learning rate, showing that architecture and other hyperparameters like learning rate can interact with each other rather than acting independently.

## 6.4 Final Tuning with Two Hidden Layers

Using the (128, 64) architecture, a few more combinations of activation, optimizer, learning rate and batch size were tried.

| Activation | Optimizer | LR    | Batch | Epochs | Accuracy | Precision | F1     |
|------------|-----------|-------|-------|--------|----------|-----------|--------|
| ReLU       | Adam      | 0.001 | 64    | 60     | 0.3578   | 0.3758    | 0.3403 |
| ReLU       | Adam      | 0.001 | 32    | 60     | 0.3358   | 0.3416    | 0.3178 |
| Tanh       | Adam      | 0.001 | 32    | 60     | 0.1848   | 0.1657    | 0.1538 |
| ReLU       | SGD       | 0.010 | 32    | 60     | 0.1642   | 0.2443    | 0.1618 |

Table 6: Hyperparameter Search with (128, 64) Architecture

**Inference:** ReLU with Adam, a learning rate of 0.001 and batch size 64 gave the best result (0.3578 accuracy), so this was chosen as the final configuration. Tanh performed noticeably worse than ReLU in every comparable setting here, and Adam consistently outperformed SGD at this stage, showing that both the activation function and optimizer choice mattered quite a lot for this dataset.

## 7 Final MLP and Justification of Chosen Hyperparameters

The training data was further split into a training and validation subset so that early stopping / validation performance could be monitored while training the final model with the chosen best configuration.

```
final_mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation="relu",
    solver="adam",
    learning_rate_init=0.001,
    batch_size=64,
    max_iter=60,
    random_state=42
)
final_mlp.fit(X_train_final, y_train_final)
```

| Parameter      | Value         |
|----------------|---------------|
| Input Features | 784           |
| Hidden Layers  | 2             |
| Neurons        | (128, 64)     |
| Activation     | ReLU          |
| Loss Function  | Cross-Entropy |
| Optimizer      | Adam          |
| Learning Rate  | 0.001         |
| Batch Size     | 64            |
| Maximum Epochs | 60            |

Table 7: Final Chosen MLP Hyperparameters

**Justification:** ReLU was chosen since it consistently outperformed Tanh in every comparable configuration tried. Adam was chosen over SGD since it converged to a better solution within the same number of epochs in both hyperparameter search stages. A learning rate of 0.001 was chosen since 0.01 consistently caused the model to collapse to near-random performance. Two hidden layers (128, 64) were chosen since the architecture search showed a large improvement over a single hidden layer.

## 8 A/B Comparison: PLA vs Tuned MLP

| Model     | Accuracy | Precision | Recall | F1-Score |
|-----------|----------|-----------|--------|----------|
| PLA       | 0.1364   | 0.1768    | 0.1364 | 0.1025   |
| Tuned MLP | 0.2903   | 0.3029    | 0.2903 | 0.2734   |

Table 8: Final A/B Comparison: PLA vs Tuned MLP

**Inference:** The tuned MLP more than doubles PLA’s accuracy (0.2903 vs 0.1364), confirming that a non-linear model with hidden layers is much better suited to this image classification task than a linear model. It is worth noting that this final test accuracy (0.2903) is a bit lower than the 0.3578 accuracy seen for the same configuration during the tuning stage, this is because the final model was trained on a smaller subset of the training data (with part of it held out as a validation set), so it saw fewer examples per class during its final training run.

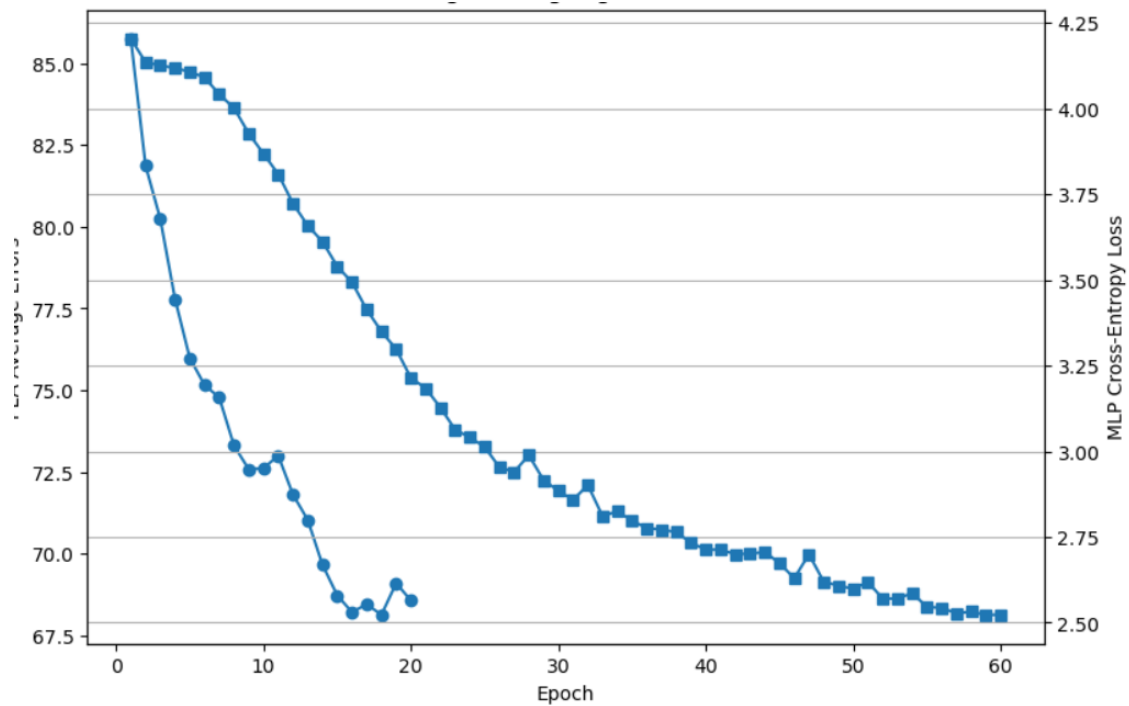


Figure 4: Training Convergence: PLA Errors vs MLP Cross-Entropy Loss

**Inference:** PLA's average error count decreases steadily but more slowly and does not reach a very low value, while the MLP's cross-entropy loss drops much faster and more smoothly, showing that gradient-based backpropagation with a non-linear model converges more efficiently on this data compared to PLA's simple mistake-driven update rule.

## 9 Confusion Matrices and ROC Curves

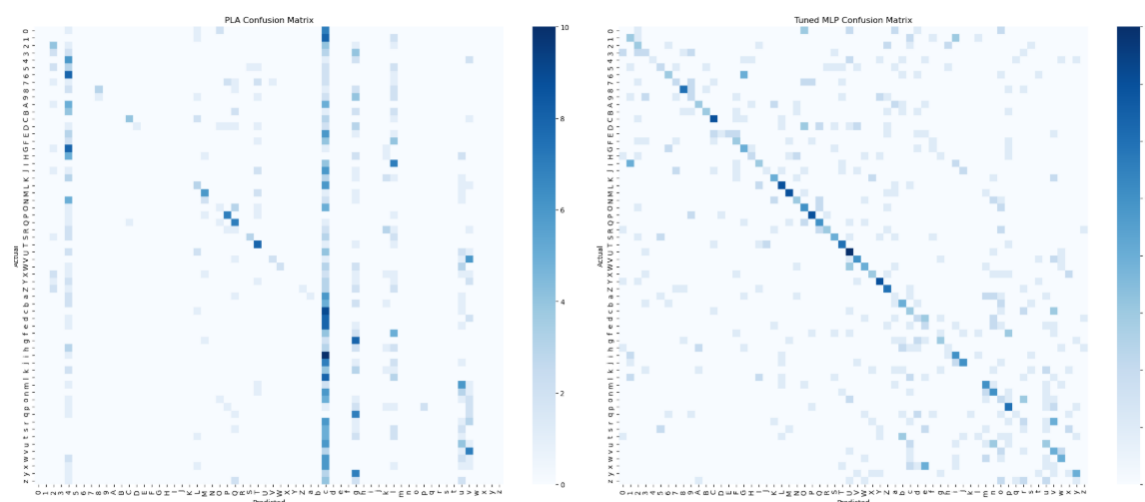


Figure 5: Confusion Matrix Comparison - PLA vs Tuned MLP

**Inference:** The PLA confusion matrix shows predictions scattered quite widely across many wrong classes, with very little concentration along the diagonal. The MLP confusion

matrix shows a noticeably stronger diagonal, meaning more of its predictions land on the correct class, even though there is still a fair amount of confusion between visually similar characters (for example between similar looking digits and letters).

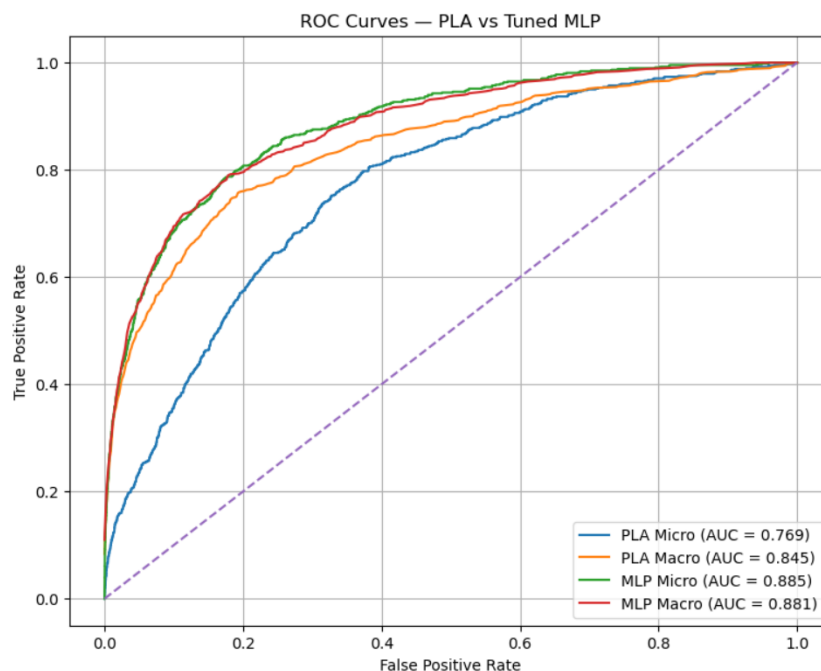


Figure 6: Micro/Macro Averaged ROC Curves - PLA vs Tuned MLP

**Inference:** Both the micro and macro averaged ROC curves for the tuned MLP lie clearly above the corresponding PLA curves, meaning the MLP is better at ranking the correct class higher across all 62 classes, not just at the default classification threshold. This is consistent with the MLP’s better accuracy, precision, recall and F1-score seen earlier.

## 10 Observation Questions

### 1. Why does PLA underperform compared to MLP?

PLA can only draw a single straight (linear) decision boundary per class, and image pixel data for 62 visually varied character classes is very unlikely to be linearly separable. MLP, on the other hand, uses hidden layers with non-linear activation functions, allowing it to build much more complex, curved decision boundaries, which is why it more than doubled PLA’s accuracy on this task.

### 2. Which hyperparameters had the most impact on MLP performance?

Learning rate had a very large impact, every configuration using  $lr=0.01$  collapsed to near-random accuracy (0.0161), while  $lr=0.001$  configurations performed much better. The number of hidden layers also had a big impact, going from one hidden layer (128,) to two hidden layers (128, 64) improved accuracy from 0.0161 to 0.3284 in the architecture search. Activation function also mattered, ReLU consistently beat Tanh in every comparable setting.

### 3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes. In the (128, 64) architecture search, the ReLU+Adam configuration reached 0.3578

accuracy while ReLU+SGD (with a larger learning rate of 0.01) only reached 0.1642 in the same 60 epochs. This suggests Adam's adaptive learning rate behaviour helped it converge to a better solution more reliably within a fixed number of epochs compared to plain SGD in this experiment.

#### **4. Did adding more hidden layers always improve results? Why or why not?**

In the architecture search, going from 1 layer to 2 layers gave a huge improvement, but this experiment did not test architectures beyond 2 hidden layers, so it cannot be concluded that more layers always helps. In general, adding more layers increases model capacity and can help learn more complex patterns, but beyond a certain point (especially with limited training data, only about 44 images per class here) it can lead to overfitting or make the model harder to optimize with the same number of training epochs.

#### **5. Did MLP show overfitting? How could it be mitigated?**

The final training loss (about 2.52) was much lower than the near-random baseline loss (about 4.13), so the model was clearly learning the training data, but the test accuracy of only 0.29 shows there is still a large gap between training and generalization performance, which points towards some degree of overfitting/underfitting balance issues given the very limited data (only about 44 training images per class for a 784-dimensional input). This could be mitigated by using data augmentation (rotations, small shifts of the character images) to effectively increase the training data, adding dropout or L2 regularization (the `alpha` parameter in scikit-learn's `MLPClassifier`), or using a convolutional neural network instead of a fully connected MLP, since CNNs are much better suited to image data with limited samples.

## **11 Conclusion**

In this experiment, a Perceptron Learning Algorithm was implemented from scratch and compared against a Multilayer Perceptron on a 62-class handwritten character recognition task. PLA achieved only 13.64% accuracy due to its inability to model non-linear decision boundaries. An untuned baseline MLP performed even worse than PLA, showing that hyperparameter tuning is essential for MLPs to work well. After systematically tuning the activation function, optimizer, learning rate, batch size and number of hidden layers, the final MLP (ReLU, Adam, learning rate 0.001, batch size 64, two hidden layers of size 128 and 64) achieved 29.03% accuracy, more than double PLA's performance. This experiment clearly demonstrated the importance of non-linearity and careful hyperparameter tuning in neural network based classifiers, while also highlighting that this particular dataset (only about 44 training images per class, 62 classes, high dimensional 784 pixel input) is genuinely challenging for both models, and would likely benefit from more training data, data augmentation, or a more specialized architecture like a CNN.

## **12 References**

1. Scikit-learn Documentation: Neural Network Models. [https://scikit-learn.org/stable/modules/neural\\_networks\\_supervised.html](https://scikit-learn.org/stable/modules/neural_networks_supervised.html)
2. English Handwritten Characters Dataset, Kaggle.